Centauri
CENTAURI HEALTH SOLUTIONS

Clinical Snapshot

Full-Stack Engineering Take-Home Assessment

Candidate Brief · Estimated effort: 24 hours

Overview

You will build a small full-stack application that ingests a messy FHIR R4 bundle for a single patient and renders a coherent, scannable clinical snapshot. The bundle is synthetic. It contains no real patient information. It has been deliberately constructed to reflect the kind of data quality problems that appear in production interoperability systems, so a large part of this exercise is deciding how to handle imperfect input safely.

We are not looking for a polished product. We are looking for sound judgment about clinical data, clean code, and evidence that you can direct an AI coding tool effectively rather than be led by it. If you are not familiar with the healthcare domain, we encourage you to use Gemini or Claude to learn about the domain.

What you are building

A patient summary view backed by a small API.

Backend (Python, FastAPI, Pydantic)

- Load the provided bundle.
- Model the resources you care about with Pydantic.
- Run a normalization pass that produces a clean, deduplicated, safe-to-display representation of the patient.
- Expose a patient-summary endpoint that the frontend consumes. You decide the shape of that response.

Frontend (TypeScript, React, Next.js)

- Render a one-page clinical snapshot: patient demographics, active problems, medications, allergies, recent encounters, and relevant observations.
- Make it scannable. A clinician should be able to read it quickly.
- Handle uncertainty visibly. Where the data is missing, ambiguous, or could not be resolved, say so rather than hiding it or guessing.

The data

You will receive a single FHIR R4 **Bundle** as JSON, containing (among others) **Patient**, **Encounter**, **Condition**, **Observation**, **MedicationRequest**, and **AllergyIntolerance** resources.

Treat this data the way you would treat production data of uncertain provenance. You should expect to encounter issues including, but not limited to:

- More than one **Patient** resource, with overlapping and partially conflicting fields.
- Resources marked with an error or inactive status that should not be presented as current clinical fact.
- Codings where the human-readable display is missing, so you have only a code and a system (for example LOINC, SNOMED CT, RxNorm, ICD-10).
- References that point to resources not present in the bundle.
- Inconsistent date precision (full timestamps in some places, year only in others).
- One or more US Core extensions.

This list is not exhaustive. Part of what we are assessing is whether you notice and handle problems that are not spelled out for you.

Using an AI coding tool

We will provide an API key with a limited budget and a specific set of enabled models. You are expected to use an AI coding assistant. We want to see how you work in a realistic, AI-assisted setting.

Include a file named **AI_USAGE.md** in your submission covering:

- Which tool and model(s) you used.
- Where it accelerated you.
- At least one specific instance where you overrode, corrected, or rejected its output, and why.

We weight the "where you corrected it" part heavily. Fluency with these tools is table stakes. Judgment about when they are wrong is the skill.

The API key provided would be for Claude, if you end up using Gemini or other model providers we require you to provide the prompts that were used to build the solution.

Scope and time

You are expected to submit the completed packaged solution to the assignment in 24 hrs. If you run short on time, prioritize correctness and safety of the data handling over breadth of features, and use your README to note what you would do next. We would rather see three things done well and one thing honestly deferred than eight things done carelessly.

What to submit

- A Git repository (or a zip of one) containing the frontend, the backend, and the provided data.
- A short **README.md** covering: how to run it, the key decisions you made, the tradeoffs you accepted under the time cap, and anything you would change with more time.
- The **AI_USAGE.md** described above.

Working setup instructions matter. If we cannot run it, we cannot evaluate it fully.

Centauri

What we are evaluating

At a high level, in rough priority order:

1. **Data handling and clinical safety.** Correct reconciliation of the messy input, and sound decisions about what is and is not safe to display as current fact.
2. **Code quality.** Clear Pydantic models, sensible typing, readable structure on both sides.
3. **Frontend clarity.** A snapshot that communicates well and handles uncertainty honestly.
4. **AI tool judgment.** Evidence you directed the tool rather than shipped its first draft unread.
5. **Communication.** A README that explains your reasoning and your tradeoffs.

Ask questions if anything is genuinely blocking. Reasonable assumptions, documented in your README, are fine and expected.